

Algorithm Cheat Sheet — All Templates

Two Pointers

Pattern 1 — Opposite Two Pointers

```
int i = 0, j = n - 1;
while (i < j) {
    while (i < j) {
        if (condition == target) {
            /* record */ i++; j--;
        } else if (tooSmall) {
            i++;
        } else {
            j--;
        }
    }
}
```

Sliding Window

The Three Templates

```
int i = 0;
for (int j = 0; j < n; j++) {
    if (j - i + 1 == k) {
        i++;
    }
}

int i = 0, result = 0;
for (int j = 0; j < n; j++) {
    while (/* window exceeds constraint */) {
        i++;
    }
    result = Math.max(result, j - i + 1);
}

int i = 0, result = Integer.MAX_VALUE;
for (int j = 0; j < n; j++) {
    while (/* window VALID */) {
        result = Math.min(result, j - i + 1);
        i++;
    }
}
```

Frequency Map Trick (used by 567 and 76)

```
if (need[s.charAt(i)]-- > 0) required--;
if (need[s.charAt(i)]++ >= 0) required++;
i++;
```

Prefix Sum + HashMap

Core Template

```
Map<Long, Integer> map = new HashMap<>();
map.put(0L, ???);
long sum = 0;
for (int i = 0; i < nums.length; i++) {
    sum += nums[i];
    long lookup = transform(sum);
    map.put(storeKey(sum), storeValue(i)); // what to store (sum or sum%k - count or index)
}
```

Binary Search

The Only Template — (i, j), find first k with condition(k)

```
int i = 0, j = nums.length;
while (i < j) {
    int k = i + (j - i) / 2;
    if (!condition(k)) {
        i = k + 1;
    } else {
        j = k;
    }
}
```

How to Choose

Sorting

Template 1 — Merge Sort

```
public void mergeSort(int[] nums) {
    mergeSort(nums, 0, nums.length, new int[nums.length]);
}

private void mergeSort(int[] nums, int start, int end, int[] temp) {
    if (end - start <= 1) return;
    int mid = start + (end - start) / 2;
    mergeSort(nums, start, mid, temp);
    mergeSort(nums, mid, end, temp);
    merge(nums, start, mid, end, temp);
}

private void merge(int[] nums, int start, int mid, int end, int[] temp) {
    int i = start, j = mid, k = start;
    while (i < mid || j < end) {
        if (j == end || (i < mid && nums[i] <= nums[j])) {
            temp[k++] = nums[i++];
        } else {
            temp[k++] = nums[j++];
        }
    }
    current.next = (a != null) ? a : b;
    return sentinel.next;
}

for (i = start; i < end; i++) nums[i] = temp[i];
}
```

Template 2 — Quick Sort (3-way Dutch Flag Partition)

```
public void quickSort(int[] nums) {
    quickSort(nums, 0, nums.length);
}

private void quickSort(int[] nums, int start, int end) {
    if (end - start <= 1) return;
    int pivot = nums[start + (end - start) / 2];
    int i = start, k = start, j = end - 1;
    while (k <= j) {
        if (nums[k] < pivot) {
            swap(nums, k, i);
            k++;
        } else if (nums[k] == pivot) {
            k++;
        } else {
            swap(nums, k, j);
            j--;
        }
    }
    quickSort(nums, start, i);
    quickSort(nums, k, end);
}

private void swap(int[] nums, int i, int j) {
    int t = nums[i]; nums[i] = nums[j]; nums[j] = t;
}
```

Template 3 — Quick Select (partial sort — k-th element)

```
private int quickSelect(int[] nums, int start, int end, int target) {
    int pivot = nums[start + (end - start) / 2];
    int i = start, k = start, j = end - 1;
    while (k <= j) {
        if (nums[k] < pivot) {
            swap(nums, k, i);
            k++;
        } else if (nums[k] == pivot) {
            k++;
        } else {
            swap(nums, k, j);
            j--;
        }
    }
    if (target < i) {
        return quickSelect(nums, start, i, target);
    } else if (target < k) {
        return pivot;
    } else {
        return quickSelect(nums, k, end, target);
    }
}
```

Template 4 — Counting Sort

```
public void countingSort(int[] arr) {
    if (arr.length == 0) return;
    int min = Arrays.stream(arr).min().getAsInt();
    int max = Arrays.stream(arr).max().getAsInt();
    int[] buckets = new int[max - min + 1];
    for (int x : arr) buckets[x - min]++;
    int i = 0;
    for (int v = 0; v < buckets.length; v++) {
        while (buckets[v]-- > 0) arr[i++] = v + min;
    }
}
```

Linked List

When to Use — Signal → Pattern

All Four Templates

```
ListNode sentinel = new ListNode(0);
sentinel.next = head;
ListNode previous = sentinel, current = head;
while (current != null) {
    if (!shouldDelete(current)) {
        previous.next = current.next;
    } else {
        previous = current;
    }
    current = current.next;
}

return sentinel.next;
ListNode previous = null, current = head;
while (current != null) {
    ListNode next = current.next;
    current.next = previous;
    previous = current;
    current = next;
}

return previous;
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}

ListNode sentinel = new ListNode(0);
ListNode current = sentinel;
while (a != null && b != null) {
    if (a.val <= b.val) {
        current.next = a;
        a = a.next;
    } else {
        current.next = b;
        b = b.next;
    }
    current = current.next;
}

current.next = (a != null) ? a : b;
return sentinel.next;
}
```

String

Core Idioms

```
int[] count = new int[26];
for (char ch : s.toCharArray()) count[ch - 'a']++;
int[] count = new int[10];
for (char ch : s.toCharArray()) count[ch - '0']++;
int num = 0;
for (int i = 0; i < s.length(); i++) {
    num = num * 10 + (s.charAt(i) - '0');
}
String hashKey(String s) {
    int[] count = new int[26];
    for (char ch : s.toCharArray()) count[ch - 'a']++;
    StringBuilder builder = new StringBuilder();
    for (int i = 0; i < 26; i++) {
        builder.append((char)('a' + i));
        builder.append(count[i]);
    }
    return builder.toString();
}

char[] chars = s.toCharArray();
Arrays.sort(chars);
String key = new String(chars);
int[] count = new int[26];
for (char ch : s.toCharArray()) count[ch]++;
List<Character>[] buckets = new List[s.length() + 1]; // index = frequency
for (int i = 0; i < 26; i++) {
    if (count[i] > 0) {
        int freq = count[i];
        if (buckets[freq] == null) buckets[freq] = new ArrayList<>();
        buckets[freq].add((char) i);
    }
}

StringBuilder builder = new StringBuilder();
for (int freq = buckets.length - 1; freq > 0; freq--) {
    if (buckets[freq] == null) continue;
    for (char ch : buckets[freq]) {
        for (int i = 0; i < freq; i++) builder.append(ch);
    }
}

int expand(String s, int i, int j) {
    while (i >= 0 && j < s.length() && s.charAt(i) == s.charAt(j)) { i--; j++; }
    return j - i - 1;
}
```

Stack / Queue / Heap

All Templates

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(x);
stack.pop();
stack.peek();
Deque<Integer> stack = new ArrayDeque<>();
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
        result[stack.pop()] = nums[i];
        stack.push(i);
    }
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && nums[stack.peek()] > nums[i]) {
            int height = nums[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
        }
        stack.push(i);
    }
    Deque<Integer> queue = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!queue.isEmpty() && nums[queue.peekLast()] <= nums[i]) {
            queue.pollLast();
            queue.offerLast(i);
        }
        if (queue.peekFirst() < i - k + 1) queue.pollFirst();
        if (i >= k - 1) result[i - k + 1] = nums[queue.peekFirst()];
    }
    PriorityQueue<Integer> minPQ = new PriorityQueue<>();
    PriorityQueue<Integer> maxPQ = new PriorityQueue<>(Collections.reverseOrder());
    // max-heap
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    custom: sort by index 1
    pq.offer(x);
    pq.poll();
    pq.peek();
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    void addNum(int num) {
        maxHeap.offer(num);
        minHeap.offer(maxHeap.poll());
        if (maxHeap.size() < minHeap.size()) {
            maxHeap.offer(minHeap.poll());
        }
    }
    double findMedian() {
        return maxHeap.size() > minHeap.size()
            ? maxHeap.peek()
            : (maxHeap.peek() + minHeap.peek()) / 2.0;
    }
}

double findMedian() {
    return maxHeap.size() > minHeap.size()
        ? maxHeap.peek()
        : (maxHeap.peek() + minHeap.peek()) / 2.0;
}
```

BFS (Tree)

The Template — Level-Aware BFS

```
Queue<TreeNode> queue = new ArrayDeque<>();
queue.offer(root);
int level = 0;
while (!queue.isEmpty()) {
    int size = queue.size();
    level++;
    for (int i = 0; i < size; i++) {
        TreeNode node = queue.poll();
        if (node.left != null) queue.offer(node.left);
        if (node.right != null) queue.offer(node.right);
    }
}
```

When to Use — Signal → Pattern

The One Rule — Snapshot Level Size First

Always snapshot the level size BEFORE the inner **for** loop:

```
int size = queue.size();
for (int i = 0; i < size; i++) { ... }
Without the snapshot, newly added children mix with the current level
and i = size-1 / i = 0 checks become meaningless.
```

DFS / Backtracking

When to Use — Signal → Pattern

All Templates

```
private int dfs(TreeNode node) {
    if (node == null) return BASE;
    int left = dfs(node.left);
    int right = dfs(node.right);
    return ...;
}

private void dfs(TreeNode node, int accumulated) {
    if (node == null) return;
    accumulated += node.val;
    if (!isLeaf(node)) { /* record */ return; }
    dfs(node.left, accumulated);
    dfs(node.right, accumulated);
}

int answer = Integer.MIN_VALUE;
private int dfs(TreeNode node) {
    if (node == null) return 0;
    int left = Math.max(0, dfs(node.left));
    int right = Math.max(0, dfs(node.right));
    answer = Math.max(answer, left + right + node.val);
    return Math.max(left, right) + node.val;
}

private void dfs(int[][] grid, int r, int c) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) return;
    if (grid[r][c] != TARGET) return;
    grid[r][c] = VISITED;
    dfs(grid, r+1, c); dfs(grid, r-1, c);
    dfs(grid, r, c+1); dfs(grid, r, c-1);
}

int[] state;
private boolean dfs(int node) {
    if (state[node] == 1) return true;
    if (state[node] == 2) return false;
    state[node] = 1;
    for (int neighbor : graph.get(node)) {
        if (dfs(neighbor)) return true;
        state[node] = 2;
    }
    return false;
}

private void backtrack(int start, List<Integer> current) {
    if (baseCase) { result.add(new ArrayList<>(current)); return; }
    for (int i = start; i < n; i++) {
        if (!skipCondition(i)) continue;
        current.add(nums[i]);
        backtrack(next, current);
        current.remove(current.size()-1);
    }
}
```

Graph

Complexity Legend

BFS / DFS	$O(V + E)$	visit each vertex and edge once
Topo sort (Kahn)	$O(V + E)$	each node enqueued once, each edge relaxed once
Union-Find	$\sim O(n \cdot \alpha(n)) = O(n)$	α = inverse Ackermann, effectively constant
Dijkstra	$O((V + E) \log V)$	non-negative weights ONLY

Graph Representation

```
List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
for (int[] edge : edges) {
    graph.get(edge[0]).add(edge[1]);
    graph.get(edge[1]).add(edge[0]);
}

List<List<int>>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
for (int[] edge : edges) {
    graph.get(edge[0]).add(new int[]{edge[1], edge[2]});
    int[] dc = {i, -1, 0};
    int[] dc = {0, 0, 1, -1};
}
```

Core Templates

```
Queue<Integer> queue = new ArrayDeque<>();
boolean[] visited = new boolean[n];
queue.offer(start);
visited[start] = true;
int level = 0;
while (!queue.isEmpty()) {
    int size = queue.size();
    for (int i = 0; i < size; i++) {
        int node = queue.poll();
        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.offer(neighbor);
            }
        }
    }
    level++;
}

int[] inDegree = new int[n];
for (int[] edge : edges) inDegree[edge[1]]++;
Queue<Integer> queue = new ArrayDeque<>();
for (int i = 0; i < n; i++) if (inDegree[i] == 0) queue.offer(i);
List<Integer> order = new ArrayList<>();
while (!queue.isEmpty()) {
    int node = queue.poll();
    order.add(node);
    for (int neighbor : graph.get(node))
        if (--inDegree[neighbor] == 0) queue.offer(neighbor);
}

int[] parent, rank;
void init(int n) {
    parent = new int[n]; rank = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
}

int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
}

boolean union(int x, int y) {
    int px = find(x), py = find(y);
    if (px == py) return false;
    if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
    parent[py] = px;
    if (rank[px] == rank[py]) rank[px]++;
    return true;
}

int[] dist = new int[n];
Arrays.fill(dist, Integer.MAX_VALUE);
dist[src] = 0;
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
pq.offer(new int[] {src, 0});
while (!pq.isEmpty()) {
    int[] current = pq.poll();
    int node = current[0], d = current[1];
    if (d > dist[node]) continue;
    for (int[] nb : graph.get(node)) {
        int next = nb[0], w = nb[1];
        if (dist[node] + w < dist[next]) {
            dist[next] = dist[node] + w;
            pq.offer(new int[] {next, dist[next]});
        }
    }
}

int[] color = new int[n];
Arrays.fill(color, -1);
Queue<Integer> queue = new ArrayDeque<>();
for (int start = 0; start < n; start++) {
    if (color[start] != -1) continue;
    color[start] = 0;
    queue.offer(start);
    while (!queue.isEmpty()) {
        int node = queue.poll();
        for (int neighbor : graph.get(node)) {
            if (color[neighbor] == -1) { color[neighbor] = 1 - color[node];
            queue.offer(neighbor); }
            else if (color[neighbor] == color[node]) return false;
        }
    }
}

return true;
}
```

Greedy

Two Interval Templates

MERGE DIRECTION **RULE** (for merge-style problems like #56, where both keys work):
Sort by **START** → traverse **FORWARD** (i = 0 → n-1), extend the **END** of last kept
Sort by **END** → traverse **BACKWARD** (i = n-1 → 0), extend the **START** of last kept
These two are exact mirror images. See #56 for both written out side by side.
(NOTE: **this** mirror only applies to merging. Template 1 below sorts by end but still sweeps **FORWARD** – there the goal is counting, not merging.)
TEMPLATE 1 – Sort by **END** time, sweep forward, track end of last kept interval
Use **when**: maximizing count of non-overlapping intervals
Greedy insight: earliest-finishing interval leaves most room for future ones
TEMPLATE 2 – Sort by **START** time, sweep forward, merge **when** overlapping
Use **when**: merging/counting overlapping intervals
Greedy insight: processing in start order → each interval only needs to compare with the last merged result

```
Arrays.sort(intervals, (a, b) -> a[1] - b[1]);
int lastEnd = Integer.MIN_VALUE;
for (int[] interval : intervals) {
    if (interval[0] >= lastEnd) {
        lastEnd = interval[1];
    } else {
    }
}
Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
List<int[]> result = new ArrayList<>();
for (int[] interval : intervals) {
    if (result.isEmpty() || result.get(result.size()-1)[1] < interval[0]) {
        result.add(interval);
    } else {
        result.get(result.size()-1)[1] = Math.max(result.get(result.size()-1)[1], interval[1]);
    }
}
Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
PriorityQueue<Integer> pq = new PriorityQueue<>();
for (int[] interval : intervals) {
    if (!pq.isEmpty() && pq.peek() <= interval[0])
        pq.poll();
    pq.offer(interval[1]);
}

Want count or min/max?
├── Count (dp[i] += dp[j - num])
│   ├── 0/1 (each item once):      j descending → #416, #494
│   ├── Unbounded (reuse):        j ascending → #518
│   └── Ordered (permut):         target outer, nums inner → #377
├── Min/Max (dp[i] = min/max(...))
│   ├── 0/1 minimize:             j descending → #474 (maximize)
│   └── Unbounded minimize:       j ascending → #322
```

Dynamic Programming

All Six Templates

```
int[] dp = new int[n + 1];
dp[0] = base;
for (int i = 1; i <= n; i++)
    dp[i] = f(dp[i-1], dp[i-2], ...);

int[] dp = new int[n];
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        dp[i] = f(dp[i], dp[j]);
    }
}
```

```
Want count or min/max?
├── Count (dp[i] += dp[j - num])
│   ├── 0/1 (each item once):      j descending → #416, #494
│   ├── Unbounded (reuse):        j ascending → #518
│   └── Ordered (permut):         target outer, nums inner → #377
├── Min/Max (dp[i] = min/max(...))
│   ├── 0/1 minimize:             j descending → #474 (maximize)
│   └── Unbounded minimize:       j ascending → #322
```

Trie

When to Use Trie (vs HashMap/Array)

Canonical Template

```
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isEnd = false;
}

class TrieNode {
    Map<Character, TrieNode> children = new HashMap<>();
    boolean isEnd = false;
}

void insert(TrieNode root, String word) {
    TrieNode node = root;
    for (char c : word.toCharArray()) {
        int idx = c - 'a';
        if (node.children[idx] == null) node.children[idx] = new TrieNode();
        node = node.children[idx];
    }
    node.isEnd = true;
}

boolean search(TrieNode root, String word) {
    TrieNode node = root;
    for (char c : word.toCharArray()) {
        int idx = c - 'a';
        if (node.children[idx] == null) return false;
        node = node.children[idx];
    }
    return node.isEnd;
}

boolean startsWith(TrieNode root, String prefix) {
    TrieNode node = root;
    for (char c : prefix.toCharArray()) {
        int idx = c - 'a';
        if (node.children[idx] == null) return false;
        node = node.children[idx];
    }
    return true;
}
```

Variations

```
boolean dfs(TrieNode node, String word, int i) {
    if (i == word.length()) return node.isEnd;
    char c = word.charAt(i);
    if (c == '.') {
        for (TrieNode child : node.children)
            if (child != null && dfs(child, word, i+1)) return true;
        return false;
    }
    int idx = c - 'a';
    if (node.children[idx] == null) return false;
    return dfs(node.children[idx], word, i + 1);
}

String findRoot(TrieNode root, String word) {
    TrieNode node = root;
    for (int i = 0; i < word.length(); i++) {
        int idx = word.charAt(i) - 'a';
        if (node.children[idx] == null) return word;
        node = node.children[idx];
        if (node.isEnd) return word.substring(0, i+1);
    }
    return word;
}

void insert(TrieNode root, String word) {
    TrieNode node = root;
    for (char c : word.toCharArray()) {
        int idx = c - 'a';
        if (node.children[idx] == null) node.children[idx] = new TrieNode();
        node = node.children[idx];
        if (node.words.size() < 3) node.words.add(word);
    }
    node.isEnd = true;
}

void dfs(char[][] board, int i, int j, TrieNode node, StringBuilder path, Set<String> result) {
    if (i < 0 || i >= board.length || j < 0 || j >= board[0].length) return;
    char c = board[i][j];
    if (c == '#' || node.children[c - 'a'] == null) return;
    node = node.children[c - 'a'];
    path.append(c);
    if (node.isEnd) result.add(path.toString());
    board[i][j] = '#';
    dfs(board, i+1, j, node, path, result);
    dfs(board, i-1, j, node, path, result);
    dfs(board, i, j+1, node, path, result);
    dfs(board, i, j-1, node, path, result);
    board[i][j] = c;
    path.deleteCharAt(path.length() - 1);
}

}
```

search vs startsWith — One Line Difference

```
search:      return node.isEnd;
startsWith:  return true;
```

Bit Manipulation

Canonical Template

```
int result = 0;
for (int num : nums) result ^= num;
boolean isSet = ((n >> 1) & 1) == 1;
int set = n | (1 << 1);
int clear = n & ~(1 << 1);
int toggle = n ^ (1 << 1);
int lowest = n & (-n);
boolean isPow2 = n > 0 && (n & (n-1)) == 0;
int count = 0;
while (n != 0) { count++; n &= (n - 1); }
int mask = 0;
for (char c : word.toCharArray()) mask |= (1 << (c - 'a'));
if ((mask[i] & mask[j]) == 0) { /* no shared letter */ }
```

Variations

```
int ones = 0, twos = 0;
for (int num : nums) {
    ones = (ones ^ num) & ~twos;
    twos = (twos ^ num) & ~ones;
}

int xor = 0;
for (int num : nums) xor ^= num;
int diff = xor & (~xor);
int a = 0;
for (int num : nums)
    if ((num & diff) != 0) a ^= num;
int b = xor ^ a;
int[] dp = new int[n + 1];
for (int i = 1; i <= n; i++)
    dp[i] = dp[i >> 1] + (i & 1);

int shift = 0;
while (left != right) { left >>= 1; right >>= 1; shift++; }
return left << shift;

int add(int a, int b) {
    while (b != 0) {
        int carry = a & b;
        a = a ^ b;
        b = carry << 1;
    }
    return a;
}
```

Design

Canonical Template — HashMap + Doubly Linked List (LRU)

```
class CacheNode {
    int key, value;
    CacheNode previous, next;
    CacheNode(int key, int value) { this.key = key; this.value = value; }
}

CacheNode head = new CacheNode(0, 0), tail = new CacheNode(0, 0);
head.next = tail; tail.previous = head;
void remove(CacheNode node) {
    node.previous.next = node.next;
    node.next.previous = node.previous;
}

void insertAfterHead(CacheNode node) {
    node.next = head.next;
    node.previous = head;
    head.next.previous = node;
    head.next = node;
}

CacheNode lru = tail.previous;
remove(lru);
map.remove(lru.key);
}
```

Variations

Math

Canonical Templates

```
while (n != 0) {
    int digit = n % 10;
    n /= 10;
}

double fastPow(double x, long n) {
    double result = 1;
    while (n > 0) {
        if ((n & 1) == 1) result *= x;
        x *= x;
        n >>= 1;
    }
    return result;
}

StringBuilder builder = new StringBuilder();
while (num > 0) {
    builder.append(num % b);
    num /= b;
}

builder.reverse();
int value = 0;
for (int digit : digits) value = value * b + digit;
boolean[] composite = new boolean[n];
for (int i = 2; i * i <= n; i++) {
    if (composite[i]) continue;
    for (int j = i * i; j <= n; j += i)
        composite[j] = true;
}
```